SC612 Assignment: Algorithms, Growth of Functions, and Complexity

Jhanvi Patel
Student ID: 202612061
Course: Discrete Mathematics

November 9, 2026

Abstract

This analytical report reviews the theoretical foundations of algorithms based on Chapter 3 of Kenneth H. Rosen's *Discrete Mathematics and Its Applications*. It examines the formal definition and fundamental properties of algorithms, surveys standard algorithmic categories, establishes asymptotic notation for the growth of functions, and evaluates algorithmic time and space complexity.

1 Introduction: What is an Algorithm?

In computational mathematics and computer science, an **algorithm** is defined as a finite sequence of unambiguous instructions for solving a well-specified computational problem or performing a specific calculation. It takes an input from a valid domain, operates deterministically or non-deterministically through sequential states, and produces an output satisfying the desired objective.

According to Rosen, any well-constructed algorithm must satisfy seven essential properties:

1. **Input:** The algorithm must take zero or more explicitly defined quantities from a specified set.
2. **Output:** The algorithm must produce one or more quantities that have a specified relation to the inputs.
3. **Definiteness:** Every individual step in the execution sequence must be clear, precise, and unambiguous.
4. **Correctness:** The algorithm must produce the correct and intended output for all valid input instances.
5. **Finiteness:** For any legitimate input, the algorithm must terminate after a finite number of discrete computational steps.
6. **Effectiveness:** Each operation must be sufficiently basic that it can, in principle, be executed exactly in a finite amount of time by a human using paper and pencil.
7. **Generality:** The procedure must be applicable to all problems of the desired form, rather than being tailored exclusively to a single, isolated set of parameters.

2 Classification and Types of Algorithms

Algorithms are categorized according to their design strategies and the structural characteristics of the problems they address:

2.1 Searching Algorithms

Searching involves locating a target value x within an ordered or unordered collection:

- **Linear Search:** Compares the target sequentially against each element from the first to the last. It operates without requiring the data to be ordered.
- **Binary Search:** Operates on a sorted sequence by comparing the search element with the median item. Based on this comparison, it halves the search interval iteratively until the element is located or the interval becomes empty.

2.2 Sorting Algorithms

Sorting arranges a sequence of elements according to an established ordering relation (e.g., numerical or lexicographical):

- **Bubble Sort:** Repeatedly traverses the list, compares adjacent elements, and swaps them if they appear in incorrect relative order.
- **Insertion Sort:** Iterates through an array, progressively taking each element and inserting it into its proper position within the already sorted prefix.

2.3 Greedy Algorithms

A greedy algorithm makes the choice that looks best at the immediate moment, optimizing locally with the objective of finding a globally optimal solution. A classical example analyzed in Chapter 3 is the *Cashier's Change-Making Algorithm*, which iteratively chooses the largest available denomination that does not exceed the remaining sum to minimize the total count of coins returned.

2.4 Brute-Force Algorithms

Brute-force algorithms evaluate candidate solutions exhaustively and systematically until a solution satisfying all formal criteria is reached or the entire domain is ruled out. While simple to implement, their combinatorial expense often limits practical feasibility on large inputs.

3 Growth of Functions

To analyze algorithm efficiency independently of specific computer architectures or compiler implementations, theoretical computer science relies on asymptotic notation.

3.1 Asymptotic Notations

- **Big-O Notation (Asymptotic Upper Bound):** Let f and g be functions from the set of integers or real numbers to the set of real numbers. We say that $f(x)$ is $O(g(x))$ if there exist positive constants C and k such that:

$$|f(x)| \leq C|g(x)| \quad \text{whenever } x > k \tag{1}$$

  The pair (C, k) are referred to as *witnesses* to the relationship.

- **Big-Ω Notation (Asymptotic Lower Bound):** We write $f(x)$ is $\Omega(g(x))$ if there exist positive constants C and k such that:

$$|f(x)| \geq C|g(x)| \quad \text{whenever } x > k \tag{2}$$

- **Big-Θ Notation (Asymptotic Tight Bound):** We say $f(x)$ is $\Theta(g(x))$ if $f(x)$ is both $O(g(x))$ and $\Omega(g(x))$. That is, there exist positive constants C_1, C_2, and k such that:

$$C_1|g(x)| \leq |f(x)| \leq C_2|g(x)| \quad \text{whenever } x > k \tag{3}$$

3.2 Hierarchy of Standard Growth Rates

As input size n approaches infinity, standard complexity functions follow this strict dominance hierarchy:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^c) < O(2^n) < O(n!) \tag{4}$$

where $c > 2$. Algorithms with polynomial bounds $O(n^c)$ are classified as tractable, whereas exponential $O(2^n)$ or factorial $O(n!)$ growth rates are classified as intractable for large instances of n.

4 Algorithmic Complexity

Algorithmic efficiency is measured across two computational resources:

1. **Time Complexity:** The total number of fundamental operations (e.g., comparisons, assignments, bit operations) executed as a function of the input size n.
2. **Space Complexity:** The total amount of physical working memory required during execution, excluding memory needed to store the original input itself.

4.1 Cases of Analysis

Because execution paths vary based on input distribution, complexity is analyzed through three cases:

- **Worst-Case Complexity:** The maximum number of operations required over all possible inputs of size n. This establishes a guaranteed theoretical performance ceiling.
- **Average-Case Complexity:** The expected number of operations averaged across all valid inputs of size n, assuming an underlying probability distribution (often uniform).
- **Best-Case Complexity:** The minimum number of operations required under the most favorable input conditions.

4.2 Complexity Summary Table

Table 1 outlines the standard time and auxiliary space complexities of classic algorithms covered in Rosen's text:

Table 1: Comparative Time and Space Complexities

Algorithm	**Best-Case Time**	**Average-Case Time**	**Worst-Case Time**	**Space Complexity**
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Matrix Multiplication	$O(n^3)$	$O(n^3)$	$O(n^3)$	$O(1)$

References

[1] Kenneth H. Rosen, *Discrete Mathematics and Its Applications*, 8th ed., New York: McGraw-Hill Education, 2019, Chapter 3: "Algorithms".

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th ed., Cambridge, MA: The MIT Press, 2022.